

Projekt pro Programování (ETE15E)

[Ariet Muzirapov](#) xmuza007@studenti.czu.cz

Název: **Zakázkovna**

1. Popis

Zakázkovna je efektivní konzolový nástroj pro freelancery, určený k evidenci zakázek, sledování příjmů a analýze plnění měsíčních cílů.

Aplikace umožňuje:

- přidávání, vyhledávání, mazání s potvrzením a zobrazení zakázek v přehledné tabulce
- výpočet celkové hodnoty a sledování procentuálního plnění cílů
- správa uživatelského profilu a systémových nastavení
- automatické ukládání dat ve formátech CSV a JSON

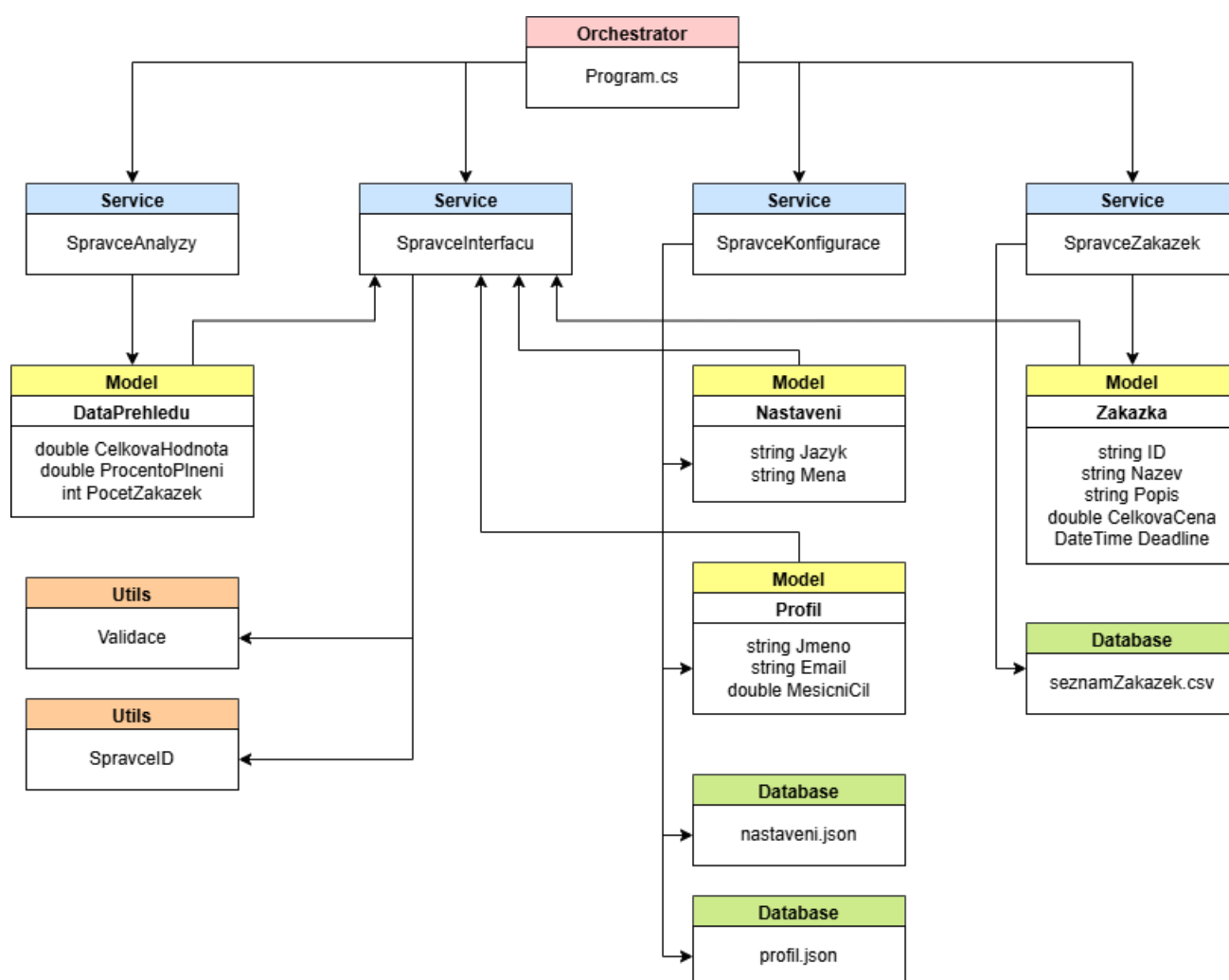
Ovládání probíhá přes intuitivní číselné menu. Integrita dat je zajištěna robustní validační vrstvou, která filtruje nekorektní vstupy (datum, cena, délka textu) před jejich zpracováním.

2. Architektura projektu

Projekt je rozdělen do pěti logických vrstev podle principu oddělení zodpovědností (každá třída dělá jen to, co jí přísluší). Díky tomu je kód přehledný a snadno rozšiřitelný.

Vrstva	Třídy	Zodpovědnost
Kořen projektu	Program.cs	hlavní smyčka, orchestrace služeb a směřování voleb
Service/	SpravceZakazek, SpravceKonfigurace, SpravceAnalyzy, SpravceInterfacu	business logika, persistence dat a komunikace s uživatelem

Models/	Zakazka, Profil, Nastaveni, DataPrehledu	datové objekty bez vnitřní logiky
Utils/	SpravceID, Validace	pomocné nástroje (generování ID a ověřování vstupů)
Database/	seznamZakazek.csv, nastaveni.json, profil.json	fyzické úložiště dat



3. Návrh hlavních proměnných a datových struktur

Níže jsou uvedeny klíčové proměnné a datové struktury, na kterých aplikace stojí.

Název	Typ	Popis
seznamZakazek	List<Zakazka>	hlavní kolekce všech zakázek načtených do operační paměti
Profil.Jmeno	string	jméno uživatele – zobrazuje se v zahlaví přehledu a nastaveních
Profil.MesicniCil	double	cílová částka pro daný měsíc - základ pro výpočet % plnění
Zakazka.ID	string	unikátní identifikátor (např. "1001"), generovaný automaticky skrze SpravceID
Zakazka.Deadline	DateTime	termín odevzdání zakázky - slouží k hlídání včasného plnění
cestaCsvSouboru	string (readonly)	cesta k úložišti dat, liší se pro vývojové (DEBUG) a ostré (RELEASE) prostředí
beziProgram	bool	řídí životní cyklus aplikace v Program.cs (false = ukončení)

Datová integrita:

Třída Zakazka má všechny vlastnosti nastaveny pouze pro čtení (get bez set). To znamená, že po vytvoření objektu není možné data zakázky omylem přepsat – změna vyžaduje smazání a vytvoření nové zakázky. Tento přístup zajišťuje integritu dat.

4. Koncepční popis programu

1. **Startování:** program inicializuje všechny správce (SpravceZakazek, SpravceInterfacu, SpravceKonfigurace, SpravceAnalyzy) a načte existující zakázky z CSV souboru do paměti.
2. **Zobrazení menu:** program čeká na volbu uživatele (číslo 0–9).
3. **Ověření volby:** vstup je ověřen třídou Validace a směrována na příslušný případ ve switch bloku.
4. **Přidání zakázky:** SpravceInterfacu zobrazí formulář a využije SpravceID k vygenerování unikátního klíče. Poté SpravceInterfacu vytvoří objekt Zakazka, který předá SpravciZakazek k uložení do kolekce a do CSV souboru.

5. **Smazání zakázky:** uživatel zadá ID, program zakázku vyhledá v seznamu pomocí LINQ. Pokud existuje, zobrazí se potvrzovací dialog. Teprve po potvrzení dojde ke smazání a přepsání CSV souboru.
6. **Přehled a analýza:** SpravceAnalyzy spočítá celkovou hodnotu zakázek (LINQ Sum), procento plnění měsíčního cíle a počet zakázek. Výsledky jsou předány do SpravceInterfacu k zobrazení.
7. **Nastavení a profil:** uživatel může upravit jméno, email, měsíční cíl, jazyk a měnu. Po návratu z nastavení program automaticky uloží veškeré změny do JSON souborů.
8. **Ukončení:** smyčka se zastaví, zobrazí se rozlučková zpráva a program skončí.

5. Ověřování vstupů a omezení

Veškeré vstupy od uživatele procházejí statickou třídou Validace před tím, než jsou jakkoliv zpracovány. Každá metoda vrací bool a výsledek předává přes out parametr – stejný vzor jako standardní TryParse metody v C#.

Pravidla ověřování:

- **Text (náý):** neprázdný, fixní min/max délka.
- **Číslo (celková cena):** rozsah 1 - 1 000 000.
- **Datum:** přesný formát DD.MM.YYYY, nesmí být v minulosti.
- **ID zakázky:** unikátní string ve formátu čtyřmístného čísla (rozsah 1000 - 9999).
- **Volba v menu:** číslo v povoleném rozsahu.

Omezení a předpoklady:

- Aplikace nepodporuje víceřádkový popis v CSV – středník a nový řádek jsou zakázané znaky v hodnotách polí.
- Maximální počet zakázek je technicky omezen pouze operační pamětí.
- Úprava existující zakázky je plánována pro verzi mimo rozsah aktuální verze – v současné verzi není implementována.
- Jazyková a měnová nastavení jsou plně funkční z hlediska správy a trvalého ukládání do JSON, avšak jejich vliv na dynamické formátování výstupů (měna u částek, lokalizace menu) je předmětem budoucí aktualizace.
- V modulech pro zápis dat (např. **Přidat zakázku**) je implementován lineární postup. Možnost návratu do hlavního menu před dokončením všech kroků není záměrně dostupná, aby se předešlo kolizi s uživatelskými vstupy (např. znak "0" jako název nebo popis). Funkce návratu (klávesa "0") je momentálně plně integrována pouze v **Hlavním menu** a v sekci **Nastavení a profil**.

6. Ukládání a načítání dat

CSV – seznam zakázek

Zakázky jsou ukládány do souboru seznamZakazek.csv ve formátu oddělených hodnot středníkem. Soubor je přepisován celý při každé změně (přidání nebo smazání). Kódování je UTF-8, takže české znaky jsou bezproblémové.

Formát řádku: **ID;Název;Popis;CelkovaCena;Deadline (dd.MM.yyyy)**

JSON – profil a nastavení

Profil uživatele a systémová nastavení jsou serializována do dvou samostatných JSON souborů pomocí System.Text.Json. Pokud soubor při spuštění neexistuje, program ho automaticky vytvoří s výchozími hodnotami. Generická metoda Nacist<T> a Ulozit<T> funguje pro oba soubory bez duplicitního kódu.

Cesty k souborům jsou odděleny direktivou **#if DEBUG / #else** – ve vývojovém prostředí míří do kořenové složky projektu, v **RELEASE** buildu do složky **Database/** vedle spustitelného souboru.

7. Prohlášení o využití AI

Při vývoji projektu Zakázkovna byly využity nástroje umělé inteligence, konkrétně **Google Gemini Pro** a **Claude 3.5 Sonnet**. AI byla použita jako podpora v následujících oblastech:

- **Návrh a revize architektury:** konzultace při návrhu a implementaci čisté struktury v prostředí C# s důrazem na oddělení logiky (Services) a dat (Models).
- **Optimalizace perzistenční vrstvy:** pomoc s návrhem generických metod Nacist<T> a Ulozit<T> pro efektivní práci s formátem JSON bez duplicity kódu.
- **Jazyková a technická korektura:** kontrola gramatické správnosti českých textů v dokumentaci a v uživatelském rozhraní aplikace.
- **Validace logiky:** skeptické posouzení navržených řešení (např. generování unikátních ID) a návrhy na ošetření chybových stavů.

Poznámka k jazyku komunikace: Vzhledem k rodnému jazyku autora probíhala pracovní komunikace s modely AI (prompty a odpovědi) v **ruském jazyce**, což umožnilo přesnější formulaci technických dotazů a hlubší pochopení architektonických konceptů. Pro účely této dokumentace byla vybraná komunikace přeložena do **českého jazyka**, aby byla zachována konzistence celého projektu.

Praktická ukázka:

Prompt (Gemini)

Jak je možné implementovat zápis/čtení nastavení a parametrů profilu? Myslím, že json je pro to vhodnější, protože csv je pohodlnější pro tabulková data

(překlad do češtiny)

Как можно реализовать запись/чтение настроек и параметров профиля? Я думаю, что для этого больше подходит json, так как csv более удобен с табличными данными

(originál v ruštině)

Odpověď (Gemini)

Hele, máš naprostou pravdu v tom, že jsi skeptický ohledně CSV pro nastavení. 🤝 CSV je skvělé pro dlouhé seznamy stejných objektů (jako tvé zakázky), ale když jde o jednotlivé objekty s různými poli (jméno uživatele, jazyk, měna), JSON je průmyslový standard.

Použití JSON ti umožní snadno profil rozšiřovat. Dnes tam máš jen jméno, zítra budeš chtít přidat pole oblíbených kategorií nebo nastavení notifikací — v JSONu je to na jeden řádek, zatímco v CSV bys musel překopávat celou tabulku.

🔧 Jak to realizovat chytře?

Aby ses vyhnul psaní samostatného kódu pro čtení profilu a nastavení, doporučuji použít **generiku (Generics)**. Tvůj `SpravceKonfigurace` tak bude krátký a elegantní. Takhle by to mohlo vypadat v kódu s využitím vestavěné knihovny `System.Text.Json`:

C#

```
using System.Text.Json;
using System.IO;

public class SpravceKonfigurace
{
    // Univerzální metoda pro uložení čehokoliv do JSON
    public void Ulozit<T>(string cesta, T data)
```

```

{
    string jsonString = JsonSerializer.Serialize(data, new
JsonSerializerOptions { WriteIndented = true });
    File.WriteAllText(cesta, jsonString);
}

// Univerzální metoda pro načtení
public T Nacist<T>(string cesta) where T : new()
{
    if (!File.Exists(cesta))
    {
        // Pokud soubor neexistuje, vrátíme prázdný objekt s výchozími
hodnotami
        return new T();
    }

    string jsonString = File.ReadAllText(cesta);
    // Pokud je soubor prázdný, také vrátíme nový objekt, aby program
nespadl
    return JsonSerializer.Deserialize<T>(jsonString) ?? new T();
}
}

```

🤖 Proč je to pro tvůj projekt skvělé řešení:

1. **Flexibilita:** Stejnou dvojicí metod načteš Profil i Nastavení.
2. **Čitelnost:** WriteIndented = true zajistí, že tvůj JSON bude hezky čitelný i v poznámkovém bloku, což je super pro ladění.
3. **Bezpečnost:** Kontrola File.Exists a vrácení new T() zajistí, že tvoje Zakázkovna nespadne hned při prvním spuštění, kdy soubory ještě nejsou ve složce.

📐 Mentorská rada k architektuře:

Snaž se rovnou oddělovat data od logiky. Tvé třídy Profil a Nastavení by měly být co nejjednodušší (jen vlastnosti), zatímco veškerou špinavou práci se soubory nechej na SpravceKonfigurace.

A nezapomeň ty soubory uložit do složky Database, ať se ti neválí v kořenu projektu. To dodá tvé struktuře složek profesionální vzhled.

Co myslíš, přidáme do nastavení i volbu grafického tématu (světlý/tmavý režim), nebo jsou to pro konzolovou aplikaci už zbytečné frajeřinky? 🤖

(překlad do češtiny)

Слушай, ты абсолютно прав в своем скепсисе относительно CSV для настроек. 🍌 CSV хорош для длинных списков одинаковых объектов (как твои заказы), но когда дело касается одиночных объектов с разным набором полей (имя пользователя, язык, валюта), JSON — это стандарт индустрии.

Использование JSON позволит тебе легко расширять профиль. Сегодня у тебя там только имя, а завтра ты захочешь добавить массив любимых категорий или настройки уведомлений — в JSON это делается одной строчкой, а в CSV тебе пришлось бы перекраивать всю «таблицу».

🔧 Как это реализовать по-умному?

Чтобы не писать отдельный код для чтения профиля и отдельный для настроек, я советую использовать **дженерики (Generics)**. Так твой `SpravceKonfigurace` будет коротким и изящным.

Вот пример того, как это может выглядеть в коде с использованием встроенной библиотеки `System.Text.Json`:

C#

```
using System.Text.Json;
using System.IO;

public class SpravceKonfigurace
{
    // Универсальный метод для сохранения чего угодно в JSON
    public void Ulozit<T>(string cesta, T data)
    {
        string jsonString = JsonSerializer.Serialize(data, new
        JsonSerializerOptions { WriteIndented = true });
        File.WriteAllText(cesta, jsonString);
    }

    // Универсальный метод для загрузки
    public T Nacist<T>(string cesta) where T : new()
    {
        if (!File.Exists(cesta))
        {
            // Если файла нет, возвращаем пустой объект с дефолтными
            значениями
            return new T();
        }

        string jsonString = File.ReadAllText(cesta);
        // Если файл пустой, тоже возвращаем новый объект, чтобы программа
        не упала
        return JsonSerializer.Deserialize<T>(jsonString) ?? new T();
    }
}
```

🤖 Почему это крутое решение для твоего проекта:

1. **Гибкость:** Одной и той же парой методов ты читаешь и `Profil`, и `Nastaveni`.
2. **Читаемость:** `WriteIndented = true` сделает твой JSON красивым и читаемым в обычном блокноте, что удобно для отладки.
3. **Безопасность:** Проверка `File.Exists` и возврат `new T()` гарантируют, что твоя «Заказовна» не вылетит с ошибкой при первом же запуске, когда файлов еще нет в папке.

🔧 Братский совет по архитектуре:

Старайся сразу разделять данные и логику. Твои классы `Profil` и `Nastaveni` должны быть максимально простыми (только свойства), а вся «грязная работа» с файлами пусть живет в `SpravceKonfigurace`.

Кстати, не забудь положить эти файлы в папку `Database`, чтобы они не валялись в корне проекта. Это сделает твою структуру папок профессиональной.

Как думаешь, стоит добавить в настройки еще и выбор темы оформления (светлая/темная), или для консольного приложения это уже лишние понты? 🤖

(originál v ruštině)

8. Plánovaný rozvoj (Roadmap)

Vzhledem k rozsahu aktuální verze MVP (Minimum Viable Product) byly definovány následující body pro budoucí aktualizace, které dále zvýší užitnou hodnotu aplikace:

- **Plná editace zakázek:** implementace funkce pro úpravu parametrů (název, cena, termín) u stávajících záznamů bez nutnosti jejich opětovného vytváření.
- **Rozšíření nastavení:** integrace zbývajících konfiguračních prvků (např. přepínání témat nebo pokročilá lokalizace menu) přímo do uživatelského rozhraní.
- **Vizuální signalizace termínů:** automatické zvýraznění "urgentních" zakázek, u kterých se blíží datum odevzdání (např. méně než 3 dny), pro lepší prioritizaci práce.